

DisrtuCare — Dossier Technique Complet

Distributeur intelligent de médicaments : du firmware ESP8266 à l'application React Native
Architecture, code commenté, flux détaillés et préparation à la maintenance

Projet DisrtuCare

Juin 2026

Table des matières

1	Présentation générale du projet	3
1.1	Qu'est-ce que DisrtuCare ?	3
1.2	Vue d'ensemble de l'architecture	3
1.3	Pile technologique	3
2	Le firmware (carte NodeMCU / ESP8266)	5
2.1	La configuration matérielle (config.h)	5
2.2	Le pilote moteur (motor.cpp)	6
2.3	L'horloge temps réel (rtc_time.cpp)	7
2.4	L'afficheur LCD (lcd_display.cpp)	8
2.5	Le programme principal (distrucare.ino)	8
3	L'application mobile (React Native / TypeScript)	12
3.1	La base de données locale (db/database.ts)	12
3.2	Les requêtes typées (db/queries.ts)	13
3.3	La communication avec le matériel (hardware/httpService.ts)	13
3.4	Le pont React (hardware/HardwareContext.tsx)	15
3.5	L'enregistrement automatique d'une prise (DashboardScreen.tsx)	16
3.6	Les notifications locales (notifications/notificationService.ts)	16
3.7	L'accessibilité (context/UISettings.tsx + theme/index.ts)	16
4	Le flux complet, étape par étape	18
4.1	Flux 1 — Premier lancement (Onboarding)	18
4.2	Flux 2 — Connexion au distributeur	18
4.3	Flux 3 — Distribution automatique d'une dose (le scénario clé)	18
4.4	Flux 4 — Modification du programme (Réglages)	19
4.5	Flux 5 — Enregistrement manuel	19
5	Nos avantages (points forts du projet)	20
5.1	Avantages techniques	20
5.2	Avantages produit / UX	20
6	Questions du jury — et leurs réponses	21
6.1	Architecture et choix généraux	21

6.2	Le matériel et le firmware	21
6.3	La communication	22
6.4	La base de données et la logique applicative	22
6.5	L'expérience utilisateur et l'accessibilité	23
6.6	Sécurité, limites et perspectives	23

7	Annexe — Référence de l'API HTTP	24
----------	---	-----------

1 Présentation générale du projet

1.1 Qu'est-ce que DisrtuCare?

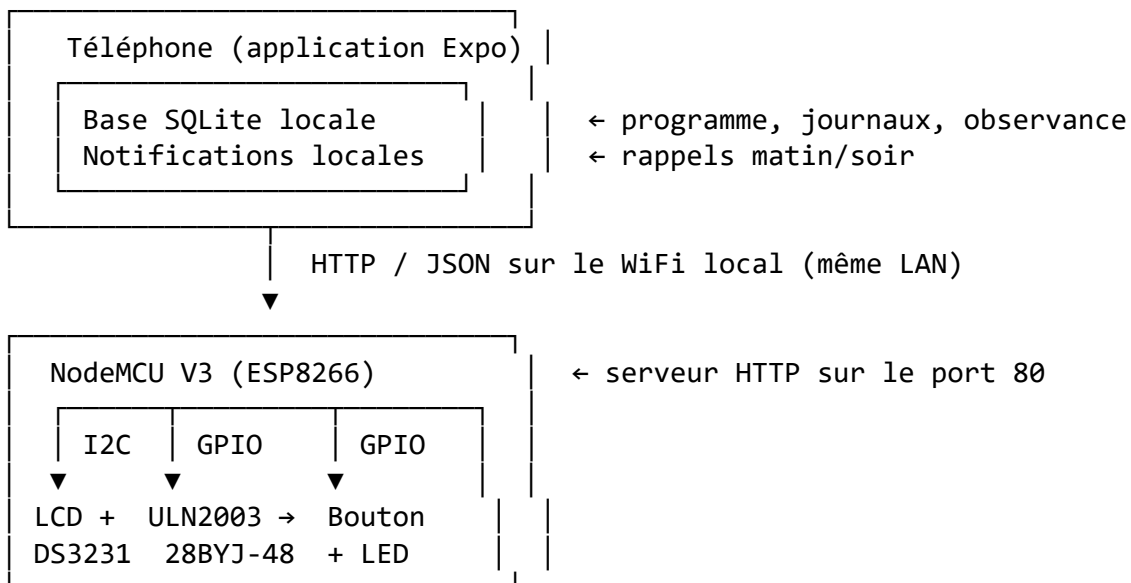
DisrtuCare est un système complet d'aide à l'observance médicamenteuse, conçu en priorité pour les **personnes âgées**. Il se compose de deux parties qui fonctionnent ensemble mais restent autonomes l'une de l'autre :

1. **Un distributeur physique de pilules** piloté par une carte **NodeMCU V3 (puce ESP8266)** : il fait tourner un container à compartiments grâce à un moteur pas-à-pas, affiche l'état sur un écran LCD, garde l'heure exacte avec une horloge temps réel, et détecte la confirmation de prise via un bouton physique.
2. **Une application mobile companion** développée en **React Native (Expo)** : elle permet de régler le programme de prise (matin / soir), de suivre l'observance (statistiques, historique), d'envoyer des rappels locaux, et de **communiquer avec le distributeur par WiFi** (HTTP).

La philosophie centrale du produit est triple :

- **Hors-ligne d'abord** (*offline-first*) : toutes les données de l'utilisateur vivent dans une base **SQLite locale** sur le téléphone. Aucun serveur cloud, aucune dépendance Internet.
- **Accessibilité** : grandes polices, contrastes forts, gros boutons, code couleur simple (vert / jaune / rouge), navigation à 2-3 taps maximum.
- **Robustesse matérielle** : le distributeur continue de fonctionner **même sans téléphone et même sans WiFi**; le téléphone n'est qu'un compagnon qui enrichit l'expérience.

1.2 Vue d'ensemble de l'architecture



Le téléphone et le NodeMCU sont sur le **même réseau WiFi domestique**. L'application connaît l'adresse IP du distributeur (affichée sur l'écran LCD au démarrage) et lui parle en **HTTP avec des messages JSON**. Il n'y a **pas de Bluetooth** et **pas de cloud**.

1.3 Pile technologique

Couche	Technologie	Rôle
Microcontrôleur	NodeMCU V3 / ESP8266 (C++ Arduino)	Pilotage matériel + serveur HTTP
Horloge	DS3231 RTC (+ horloge logicielle de secours)	Heure exacte pour déclencher les doses
Affichage	LCD 16×2 via bus I2C (PCF8574)	Retour visuel sur le distributeur
Actionneur	Moteur pas-à-pas 28BYJ-48 + driver ULN2003	Rotation du container de pilules
Application	React Native 0.81 + Expo 54 + TypeScript	Interface utilisateur mobile
Base de données	expo-sqlite (SQLite)	Stockage local hors-ligne
Notifications	expo-notifications	Rappels locaux
Communication	fetch() HTTP / JSON	Liaison app ↔ distributeur

2 Le firmware (carte NodeMCU / ESP8266)

Le firmware est écrit en C++ (style Arduino) et organisé en modules. Tous les fichiers vivent dans `firmware/distruccare/`.

Fichier	Rôle
<code>config.h</code>	Brochage (pins), constantes moteur, carte EEPROM, port HTTP
<code>motor.cpp/.h</code>	Pilote demi-pas du moteur 28BYJ-48 + auto-test
<code>lcd_display.cpp/.h</code>	Affichage LCD via I2C (auto-détection)
<code>rtc_time.cpp/.h</code>	Horloge DS3231 + horloge logicielle de secours
<code>distruccare.ino</code>	Programme principal : WiFi, serveur HTTP, logique de planification

2.1 La configuration matérielle (`config.h`)

Ce fichier centralise toute la configuration matérielle. Le point le plus important — et un vrai morceau d'ingénierie de débogage — est le **choix des broches**.

```
// Stepper motor → ULN2003 IN1-IN4
#define MOTOR_IN1 D5 // GPIO14
#define MOTOR_IN2 D6 // GPIO12
#define MOTOR_IN3 D7 // GPIO13
// IN4 déplacé HORS de D8 (GPIO15, broche de "strapping" qui maintenait la
// puce en mode arrêt au démarrage) vers D0 (GPIO16) – broche sûre, non-strapping.
#define MOTOR_IN4 D0 // GPIO16 (anciennement D8)

// Bus I2C – partagé par l'afficheur LCD (PCF8574) et l'horloge DS3231
#define I2C_SDA D2 // GPIO4
#define I2C_SCL D1 // GPIO5

#define LCD_ADDRESS 0x27 // adresse I2C du backpack LCD (ou 0x3F)
#define BUTTON_PIN D3 // GPIO0, INPUT_PULLUP (bouton FLASH intégré)
```

Pourquoi c'est important : sur l'ESP8266, certaines broches (GPIO0, GPIO2, GPIO15) sont des **broches de strapping** : leur niveau électrique au moment exact de la mise sous tension décide du mode de démarrage de la puce. À l'origine, la 4^e bobine du moteur (IN4) était câblée sur **D8 (GPIO15)**. Le driver ULN2003 maintenait cette ligne légèrement haute, ce qui plaçait l'ESP8266 en **mode arrêt** au boot : la puce ne démarrait pas et ne rejoignait jamais le WiFi. La solution a été de **déplacer IN4 sur D0 (GPIO16)**, une broche sûre. C'est typiquement le genre de problème qu'un jury technique apprécie d'entendre expliquer.

2.1.1 Constantes du moteur (calcul de la rotation)

```
// 28BYJ-48 : 2048 pas complets/tour -> 4096 DEMI-pas par tour du moteur.
// Réducteur 7,5:1 entraînant un container à 15 compartiments :
// STEPS_PER_SLOT = 4096 * (7,5 / 15) = 2048 demi-pas (= 1/2 tour)
#define HALFSTEPS_PER_REV 4096
```

```
#define STEPS_PER_SLOT    2048    // un compartiment = un demi-tour exactement
#define STEP_DELAY_MS      2      // 2 ms/demi-pas -> ~4,1 s par compartiment
```

Pour avancer le container **d'un compartiment**, le firmware envoie exactement 2048 demi-pas. Cette valeur se recalcule simplement si l'on change la mécanique (engrenage ou nombre de compartiments).

2.1.2 Carte EEPROM (persistance)

```
#define EEPROM_SIZE      60
#define EEPROM_AM_HOUR   0
#define EEPROM_AM_MIN    1
#define EEPROM_PM_HOUR   2
#define EEPROM_PM_MIN    3
#define EEPROM_MED_NAME  4    // 50 octets, indices 4-53
#define EEPROM_VALID     54    // 0xAB = données initialisées
#define EEPROM_MAGIC     0xAB
```

Le programme (heures matin/soir + nom du médicament) est sauvegardé dans l'EEPROM simulée de l'ESP8266. Un **octet « magique » (0xAB)** sert de sentinelle : s'il n'est pas présent, le firmware sait que l'EEPROM n'a jamais été initialisée et garde ses valeurs par défaut, évitant de lire des données aléatoires.

2.2 Le pilote moteur (motor.cpp)

Le moteur 28BYJ-48 est piloté en **demi-pas (half-step)**, une séquence de 8 micro-états qui offre plus de couple et une rotation plus fluide que le pas complet.

```
// Séquence demi-pas pour le 28BYJ-48 – 8 micro-pas par cycle électrique.
static const uint8_t HALF_STEP[8][4] = {
    {1, 0, 0, 0},
    {1, 1, 0, 0},
    {0, 1, 0, 0},
    {0, 1, 1, 0},
    {0, 0, 1, 0},
    {0, 0, 1, 1},
    {0, 0, 0, 1},
    {1, 0, 0, 1},
};

static uint8_t stepIndex = 0;
static const uint8_t PINS[4] = { MOTOR_IN1, MOTOR_IN2, MOTOR_IN3, MOTOR_IN4 };

static void applyStep() {
    for (uint8_t i = 0; i < 4; i++) {
        digitalWrite(PINS[i], HALF_STEP[stepIndex][i]);
    }
    stepIndex = (stepIndex + 1) % 8;    // avance dans le cycle, en boucle
    delay(STEP_DELAY_MS);
}
```

```

void motorRotate(int slots) {
    long totalSteps = (long)slots * STEPS_PER_SLOT;
    for (long i = 0; i < totalSteps; i++) {
        applyStep();
    }
    motorStop();    // coupe les bobines pour ne pas chauffer/consommer
}

void motorStop() {
    for (uint8_t i = 0; i < 4; i++) digitalWrite(PINS[i], LOW);
}

```

Comment ça marche : chaque appel à `applyStep()` écrit le motif suivant de la séquence sur les 4 broches, ce qui fait avancer le rotor d'un demi-pas. `motorRotate(slots)` répète l'opération `slots` × 2048 fois, puis **coupe l'alimentation des bobines** (`motorStop`) — étape cruciale : laisser une bobine alimentée à l'arrêt la ferait chauffer inutilement et consommerait du courant.

Détail anti-brownout important : l'auto-test moteur n'est **volontairement pas** lancé au démarrage. Faire tourner le moteur au boot provoque un appel de courant (inrush) qui peut faire chuter la tension et empêcher l'ESP8266 de rejoindre le WiFi. L'auto-test est donc exposé **à la demande** via l'endpoint `/selftest`.

2.3 L'horloge temps réel (`rtc_time.cpp`)

Le firmware gère deux sources de temps avec **repli automatique (fallback)** :

- Si une puce **DS3231** est détectée sur le bus I2C → on l'utilise (elle garde l'heure même hors tension grâce à sa pile).
- Sinon → on utilise une **horloge logicielle** : l'application envoie l'heure via `/settime`, et le firmware l'avance avec `millis()`.

```

void rtcInit() {
    if (rtc.begin()) {
        hwPresent = true;
        if (rtc.lostPower()) {
            // Repli sur l'heure de compilation jusqu'à ce que l'app synchronise
            rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
        }
    } else {
        hwPresent = false;    // utilisera l'horloge logicielle dès /settime
    }
}

// "Running" = on connaît l'heure assez bien pour planifier une dose.
bool rtcIsRunning() {
    return hwPresent || swValid;
}

uint8_t rtcHour() {
    if (hwPresent) return rtc.now().hour();
}

```

```

    if (swValid)    return (uint8_t)((swSecondsOfDay() / 3600UL) % 24UL);
    return 0;
}

```

Pourquoi ce design est élégant : le système fonctionne **avec ou sans** la puce RTC physique. Sans RTC, l'app resynchronise l'heure à chaque connexion. La fonction `rtcIsRunning()` empêche un piège classique : déclencher une dose à « 00 :00 » alors que l'heure n'est en réalité pas encore connue.

2.4 L'afficheur LCD (`lcd_display.cpp`)

Le LCD est **optionnel et auto-détecté**. Le code sonde le bus I2C aux adresses 0x27 puis 0x3F; si rien ne répond, **tous** les appels LCD sont ignorés silencieusement.

```

void lcdInit() {
    uint8_t addr = 0;
    if      (i2cHas(0x27)) addr = 0x27;
    else if (i2cHas(0x3F)) addr = 0x3F;
    if (addr == 0) { lcdPresent = false; return; } // pas de LCD -> on continue

    lcd = new LiquidCrystal_I2C(addr, LCD_COLS, LCD_ROWS);
    lcdPresent = true;
    lcd->init();
    lcd->backlight();
    lcd->setCursor(2, 0); lcd->print("DisrtuCare");
}

void lcdShowSchedule(const char* am, const char* pm) {
    if (!lcdPresent) return; // garde de sûreté systématique
    lcd->clear();
    lcd->setCursor(0, 0); lcd->print("AM: "); lcd->print(am);
    lcd->setCursor(0, 1); lcd->print("PM: "); lcd->print(pm);
}

```

Pourquoi c'est robuste : historiquement, un LCD absent ou défectueux **bloquait le `setup()`**, empêchant le serveur HTTP de démarrer. En rendant le LCD facultatif, on garantit que le cœur du système (WiFi + HTTP + moteur) fonctionne quoi qu'il arrive.

2.5 Le programme principal (`distrucare.ino`)

C'est le chef d'orchestre. Il gère le WiFi, le serveur HTTP, la logique de planification, le bouton et la LED.

2.5.1 Connexion WiFi multi-réseaux

```

static ESP8266WiFiMulti wifiMulti;
static void setupWifiList() {
    wifiMulti.addAP("MG1", "doc.client@123");
    wifiMulti.addAP("La_Fibre_dOrange_0E96", "DQKFSQCZ3QCHRAUTT7");
    wifiMulti.addAP("iPhone", "aya123456789");
}

```


Le distributeur connaît **plusieurs réseaux** et rejoint automatiquement le plus fort disponible. On peut donc le déplacer entre la maison, un point d'accès téléphone, etc., **sans reflasher**. La fonction `mainWifi()` dans la boucle reconnecte automatiquement si le réseau tombe.

2.5.2 Le serveur HTTP et ses routes

```
server.on("/status", HTTP_GET, handleStatus); // état complet (polling)
server.on("/sync", HTTP_POST, handleSync); // mettre à jour le programme
server.on("/settime", HTTP_POST, handleSetTime); // régler l'horloge
server.on("/led", HTTP_POST, handleLed); // allumer/éteindre la LED
server.on("/motor", HTTP_POST, handleMotor); // tourner d'un cran
server.on("/diag", HTTP_GET, handleDiag); // diagnostic matériel
server.on("/selftest", HTTP_GET, handleSelfTest); // auto-test moteur
```

2.5.3 L'endpoint /status (cœur de la communication)

L'application interroge cet endpoint **toutes les 3 secondes**. Il renvoie tout l'état du distributeur en JSON, y compris un **compteur d'événements** (`event_id`).

```
static void handleStatus() {
    cors();
    char buf[320];
    snprintf(buf, sizeof(buf),
        "{\"am\":\"%s\", \"pm\":\"%s\", \"med\":\"%s\", \"",
        "\"time\":\"%s\", \"rtc\":\"%s\", \"awaiting\":\"%s\", \"",
        "\"event_id\":%u, \"last_event\":\"%s\"}",
        amStr, pmStr, medName,
        rtcTimeString().c_str(),
        rtcIsRunning() ? "true" : "false",
        awaitingConfirm ? "true" : "false",
        (unsigned)eventId,
        lastEventJson
    );
    server.send(200, "application/json", buf);
}
```

Le mécanisme d'événements (pattern clé) : plutôt que de pousser des notifications vers le téléphone (compliqué et fragile), le firmware maintient un compteur `event_id` et le dernier événement `last_event`. Chaque fois qu'il se passe quelque chose (dose distribuée, bouton pressé), `event_id` augmente. L'application, en interrogeant `/status`, **détecte le changement de `event_id`** et déclenche l'action correspondante. C'est un modèle de *polling* simple, sans état côté serveur, parfaitement adapté à un microcontrôleur.

```
static void pushEvent(const char* json) {
    eventId++; // L'app détecte ce changement
    strncpy(lastEventJson, json, sizeof(lastEventJson) - 1);
    lastEventJson[sizeof(lastEventJson) - 1] = '\0';
}
```

2.5.4 La distribution automatique d'une dose

```
static void dispense(const char* type) {
    lcdShowDispensing();
    motorRotate(1);           // avance d'un compartiment
    lcdShowDone();

    ledBlinking    = true;    // la LED clignote pour alerter
    awaitingConfirm = true;    // on attend l'appui sur le bouton
    strncpy(pendingType, type, 2);

    char evt[64];
    snprintf(evt, sizeof(evt), "{\"evt\":\"dispensed\",\"type\":\"%s\"}", type);
    pushEvent(evt);           // notifie l'application
}
```

2.5.5 La planification (vérifiée à chaque tour de boucle)

```
static void checkSchedule() {
    if (!rtcIsRunning()) return;

    uint16_t nowMins = rtcHour() * 60 + rtcMinute();
    uint16_t amMins  = amHour  * 60 + amMin;
    uint16_t pmMins  = pmHour  * 60 + pmMin;

    if (nowMins == amMins && lastDispensedMin != (int)amMins) {
        lastDispensedMin = (int)amMins;
        dispense("AM");
    } else if (nowMins == pmMins && lastDispensedMin != (int)pmMins) {
        lastDispensedMin = (int)pmMins;
        dispense("PM");
    }

    // Réinitialisation quotidienne à minuit pour que demain redéclenche
    if (nowMins == 0 && lastDispensedMin > 0) lastDispensedMin = -1;
}
```

Logique anti-double-distribution : lastDispensedMin mémorise la minute de la dernière distribution. Comme checkSchedule() est appelée des milliers de fois par minute, sans ce garde-fou la dose serait distribuée en continu pendant toute la minute concernée. À minuit, le compteur est remis à zéro pour le lendemain.

2.5.6 Le bouton physique (anti-rebond / debounce)

```
static void checkButton() {
    bool reading = digitalRead(BUTTON_PIN);
    unsigned long now = millis();
```

```

if (reading != lastBtnReading) lastDebounceMs = now;
lastBtnReading = reading;
if (now - lastDebounceMs < 50) return;    // ignore les rebonds < 50 ms

if (reading == LOW && awaitingConfirm) {
    awaitingConfirm = false;
    ledBlinking     = false;
    digitalWrite(LED_PIN, HIGH); delay(1500); digitalWrite(LED_PIN, LOW);

    char evt[48];
    snprintf(evt, sizeof(evt), "{\"evt\":\"confirmed\",\"type\":\"%s\"}", pendingType);
    pushEvent(evt);           // l'app enregistrera la dose comme "prise"
}
}

```

Un bouton mécanique « rebondit » (plusieurs micro-contacts en quelques millisecondes). Le code ignore tout changement survenu moins de 50 ms après le précédent : c'est l'**anti-rebond logiciel**. Quand l'appui valide est confirmé pendant une attente, le firmware émet l'événement confirmed.

2.5.7 La boucle principale

```

void loop() {
    server.handleClient();    // traite les requêtes HTTP entrantes
    maintainWifi();           // reconnecte le WiFi si besoin
    checkSchedule();           // est-il l'heure d'une dose ?
    checkButton();             // l'utilisateur a-t-il confirmé ?
    updateLed();               // fait clignoter la LED en attente
}

```

Tout est **non bloquant** : aucune fonction n'attend longtemps, ce qui garde le serveur HTTP réactif tout en surveillant le temps, le bouton et la LED.

3 L'application mobile (React Native / TypeScript)

L'application est une application **Expo** organisée en couches claires : base de données, hooks, contextes, service matériel, écrans et composants.

```
src/
├── db/                database.ts, queries.ts      (SQLite)
├── hooks/             useSchedule.ts, useLogs.ts   (logique de données)
├── context/           UISettings.tsx              (accessibilité)
├── hardware/          httpService.ts, HardwareContext.tsx
├── notifications/     notificationService.ts
├── screens/           Dashboard, History, Settings, Hardware, Onboarding
├── components/        DoseCard, AdherenceRing, StatusBadge, ConfirmModal
├── theme/             index.ts                   (palette, polices)
└── utils/            dateHelpers.ts
```

3.1 La base de données locale (db/database.ts)

Tout est stocké en **SQLite** sur le téléphone. Deux tables : schedule (le programme et les préférences) et dose_logs (le journal des prises).

```
await database.execAsync(`
  PRAGMA journal_mode = WAL;

  CREATE TABLE IF NOT EXISTS schedule (
    id INTEGER PRIMARY KEY NOT NULL,
    medication_name TEXT NOT NULL DEFAULT 'My Medication',
    am_time TEXT NOT NULL DEFAULT '08:00',
    pm_time TEXT NOT NULL DEFAULT '20:00',
    notifications_enabled INTEGER NOT NULL DEFAULT 1,
    is_setup INTEGER NOT NULL DEFAULT 0,
    device_ip TEXT NOT NULL DEFAULT '',
    large_text INTEGER NOT NULL DEFAULT 0
  );

  CREATE TABLE IF NOT EXISTS dose_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    date TEXT NOT NULL,
    dose_type TEXT NOT NULL CHECK(dose_type IN ('AM', 'PM')),
    status TEXT NOT NULL CHECK(status IN ('taken', 'missed', 'late', 'pending')),
    logged_at TEXT NOT NULL,
    UNIQUE(date, dose_type)
  );
`);
```

Points de conception notables :

- PRAGMA journal_mode = WAL (Write-Ahead Logging) améliore les performances et la fiabilité des écritures concurrentes.
- Les contraintes CHECK(...) garantissent l'**intégrité** : un statut ne peut être que taken/missed/late/pending.

- La contrainte `UNIQUE(date, dose_type)` garantit **une seule entrée par dose et par jour** — elle rend possible le pattern *upsert* (insérer ou mettre à jour).
- Des migrations défensives (`ALTER TABLE ...` dans des `try/catch`) ajoutent les colonnes récentes sur les anciennes installations sans casser les données existantes.

3.2 Les requêtes typées (`db/queries.ts`)

Le pattern **upsert** est au cœur de l'enregistrement d'une dose :

```
export async function upsertDoseLog(
  date: string,
  doseType: 'AM' | 'PM',
  status: 'taken' | 'missed' | 'late'
): Promise<void> {
  const db = await getDatabase();
  const loggedAt = new Date().toISOString();
  await db.runAsync(
    `INSERT INTO dose_logs (date, dose_type, status, logged_at)
    VALUES (?, ?, ?, ?)
    ON CONFLICT(date, dose_type) DO UPDATE SET
      status = excluded.status, logged_at = excluded.logged_at`,
    [date, doseType, status, loggedAt]
  );
}
```

`ON CONFLICT ... DO UPDATE` : si une entrée existe déjà pour ce jour et cette dose, elle est **mise à jour**; sinon elle est **créée**. L'utilisateur peut donc corriger un statut (de « manquée » à « prise ») sans créer de doublon.

Le calcul de l'**observance** (taux d'adhésion) :

```
export async function getAdherenceStats(days: number = 30) {
  // ... agrège les statuts sur la période
  const total = taken + missed + late;
  const percentage = total > 0
    ? Math.round(((taken + late) / total) * 100) // "en retard" compte comme pris
    : 0;
  return { total, taken, missed, late, percentage };
}
```

Décision produit : une dose « en retard » est comptée comme **prise** dans le pourcentage d'observance, car le médicament a bien été ingéré. Seules les doses « manquées » pénalisent le score. Toutes les requêtes utilisent des **requêtes paramétrées** (?), ce qui élimine tout risque d'injection SQL.

3.3 La communication avec le matériel (`hardware/httpService.ts`)

C'est le **singleton** qui gère toute la liaison HTTP avec le distributeur. Il expose : connexion, déconnexion, envoi de commandes, et un système d'abonnement aux événements.

```
export type HardwareEvent =
  | { evt: 'dispensed'; type: 'AM' | 'PM' | 'TEST' }
  | { evt: 'confirmed'; type: 'AM' | 'PM' | 'TS' }
```

```
| { evt: 'error';      msg: string };

const POLL_MS      = 3_000;
const TIMEOUT_MS   = 6_000;
```

Les **types discriminés** (*discriminated unions*) de TypeScript décrivent exactement les événements possibles : le compilateur garantit qu'on ne lit msg que sur un événement error, etc.

3.3.1 La connexion (avec synchronisation de l'heure)

```
async connect(): Promise<void> {
  if (!this.baseUrl) throw new Error('Aucune IP. Saisissez celle affichée sur le LCD.');
```

```
  this.emitStatus('connecting');
```

```
  try {
    const resp = await fetch(`${this.baseUrl}/status`, { signal: withTimeout(TIMEOUT_MS) });
    if (!resp.ok) throw new Error('Réponse invalide du périphérique.');
```

```
  } catch (e: any) {
    this.emitStatus('disconnected');
```

```
    throw new Error('Périphérique injoignable. Vérifiez l\'IP et le WiFi.');
```

```
  }
```

```
  // Synchronise immédiatement l'heure locale vers l'horloge du distributeur
  const now = new Date();
  await fetch(`${this.baseUrl}/settime`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      h: now.getHours(), m: now.getMinutes(), s: now.getSeconds(),
      day: now.getDate(), month: now.getMonth() + 1, year: now.getFullYear(),
    }),
  }).catch(() => { /* non bloquant */ });

  this.emitStatus('connected');
```

```
  this.startPolling();
}
```

À la connexion, l'app **pousse l'heure du téléphone** vers le distributeur. Ainsi, même sans puce RTC, le distributeur a toujours l'heure exacte. Chaque fetch a un **timeout** : si le distributeur ne répond pas en 6 s, on bascule proprement en « déconnecté ».

3.3.2 Le polling (détection d'événements)

```
private startPolling(): void {
  this.pollTimer = setInterval(async () => {
    try {
      const resp = await fetch(`${this.baseUrl}/status`, { signal: withTimeout(TIMEOUT_MS) });
      const data = await resp.json();
```

```

    // Un nouvel event_id => un nouvel événement matériel à traiter
    if (typeof data.event_id === 'number' && data.event_id !== this.lastEventId) {
        this.lastEventId = data.event_id;
        if (data.last_event) {
            this.eventHandlers.forEach((h) => h(data.last_event as HardwareEvent));
        }
    }
} catch {
    this.stopPolling();
    this.emitStatus('disconnected'); // perte de réseau détectée automatiquement
}
}, POLL_MS);
}

```

C'est le **pendant côté app** du mécanisme event_id du firmware : on compare l'event_id reçu au dernier connu; s'il a changé, on notifie tous les abonnés. Si une requête échoue, on en déduit une **perte de connexion** et on met à jour l'état en conséquence.

3.3.3 Le pattern observateur (abonnements)

```

onEvent(handler: EventHandler): () => void {
    this.eventHandlers.add(handler);
    return () => this.eventHandlers.delete(handler); // fonction de désabonnement
}

```

Chaque abonnement renvoie une **fonction de nettoyage**, parfaitement adaptée au useEffect de React (qui appelle cette fonction au démontage du composant). Cela évite les fuites mémoire.

3.4 Le pont React (hardware/HardwareContext.tsx)

Ce contexte expose le service HTTP à toute l'application via les hooks React, et **persiste l'adresse IP** du distributeur dans SQLite.

```

useEffect(() => {
    // Recharge l'IP mémorisée et enregistre les écouteurs du service
    getDeviceIp().then((ip) => {
        if (ip) { httpService.setUrl(ip); setSavedIp(ip); }
    });

    const offStatus = httpService.onStatus(setStatus);
    const offEvent = httpService.onEvent(setLastEvent);
    return () => { offStatus(); offEvent(); }; // nettoyage au démontage
}, []);

const syncSchedule = useCallback(async (s: ScheduleLike) => {
    await httpService.post('sync', { am: s.am_time, pm: s.pm_time, med: s.medication_name });
}, []);

```

L'IP étant mémorisée en base, l'utilisateur **n'a pas à la ressaisir** à chaque ouverture de l'app : c'est un

vrai confort pour une personne âgée.

3.5 L'enregistrement automatique d'une prise (DashboardScreen.tsx)

Voici le moment où le matériel et le logiciel se rejoignent : quand l'utilisateur appuie sur le **bouton physique** du distributeur, l'application l'enregistre **automatiquement**.

```
useEffect(() => {
  if (!lastEvent || lastEvent === prevHwEvent.current) return;
  prevHwEvent.current = lastEvent;

  if (lastEvent.evt === 'confirmed' && (lastEvent.type === 'AM' || lastEvent.type === 'PM')) {
    logDose(lastEvent.type, 'taken'); // écrit en base SQLite
    const t = lastEvent.type === 'AM' ? 'du matin' : 'du soir';
    Alert.alert('Prise confirmée',
      `La dose ${t} a été enregistrée automatiquement depuis le distributeur.`);
  }
}, [lastEvent, logDose]);
```

L'utilisateur n'a **rien à faire dans l'app** : il prend sa pilule, appuie sur le bouton, et la prise est journalisée et affichée. C'est le cœur de l'expérience « companion ».

3.6 Les notifications locales (notifications/notificationService.ts)

Quatre rappels quotidiens sont programmés : matin, soir, et deux **relances 30 minutes après** chaque dose si elle n'a pas été confirmée.

```
export async function scheduleDoseReminders(amTime: string, pmTime: string, medName: string) {
  await Notifications.cancelAllScheduledNotificationsAsync(); // repart propre
  const [amHour, amMin] = amTime.split(':').map(Number);

  await Notifications.scheduleNotificationAsync({
    content: { title: 'Dose du matin', body: `C'est l'heure de prendre votre ${medName}.` },
    trigger: { type: Notifications.SchedulableTriggerInputTypes.DAILY, hour: amHour, minut
  });
  // ... + dose du soir + 2 relances 30 min après
}
```

Tout repose sur des déclencheurs **quotidiens** (DAILY) gérés par le système d'exploitation : les rappels fonctionnent **sans Internet** et même application fermée.

3.7 L'accessibilité (context/UISettings.tsx + theme/index.ts)

Un mode « **confort visuel** » agrandit **toute** la typographie d'un facteur 1,32. Le réglage est persisté en base et appliqué globalement.

```
export const FontScale = { normal: 1, large: 1.32 };

export function makeFonts(large: boolean): Fonts {
  const k = large ? FontScale.large : FontScale.normal;
  return {
```



```
xs: Math.round(BaseFontSizes.xs * k),    // 13 -> 17
md: Math.round(BaseFontSizes.md * k),    // 17 -> 22
xxx1: Math.round(BaseFontSizes.xxx1 * k),
// ...
};
}
```

Chaque écran reconstruit ses styles à partir de ces tailles via `useMemo(() => makeStyles(fonts), [fonts])`. Le moindre texte de l'application grandit d'un seul appui — essentiel pour le public cible.

4 Le flux complet, étape par étape

4.1 Flux 1 — Premier lancement (Onboarding)

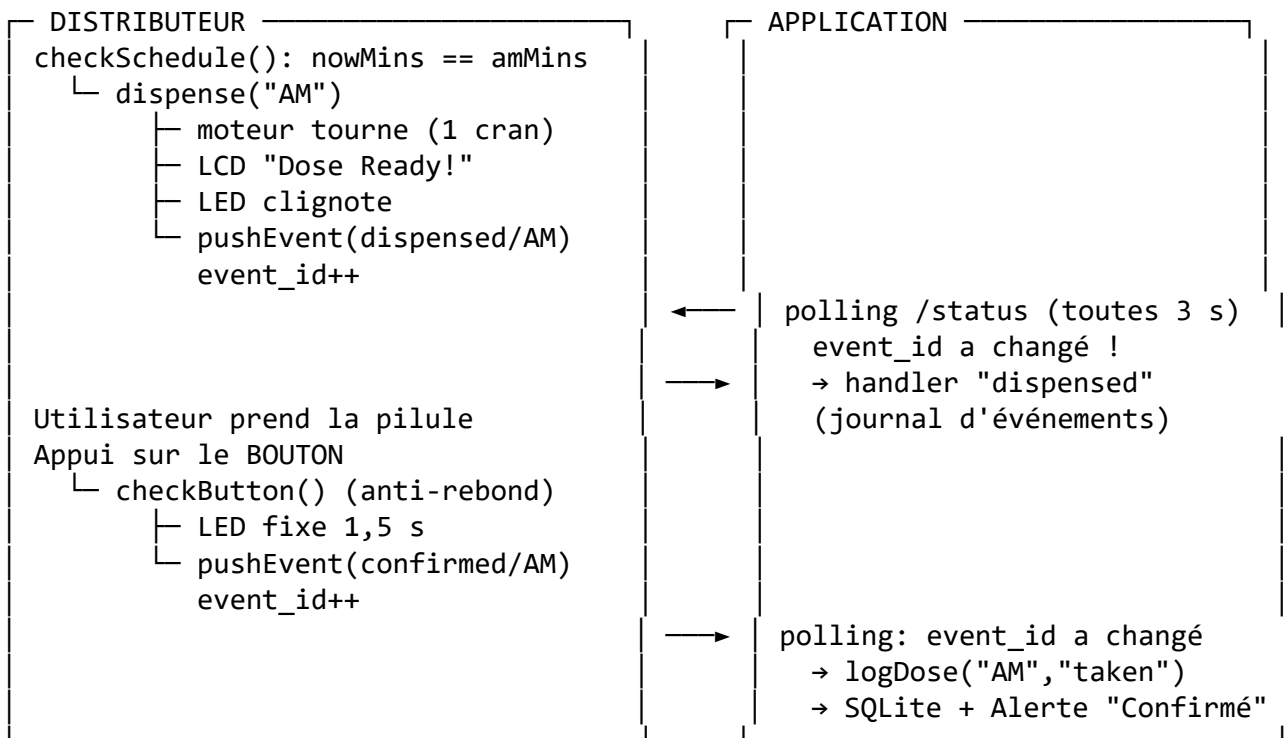
1. App.tsx initialise la base SQLite (initDatabase) puis lit is_setup.
2. Si l'utilisateur n'a jamais configuré l'app, l'écran **Onboarding** s'affiche en 4 étapes : bienvenue → nom du médicament → horaires matin/soir → terminé.
3. À la fin, updateSchedule(...) écrit le programme, les permissions de notification sont demandées, et markSetupComplete() bascule is_setup = 1.
4. Les lancements suivants vont directement au tableau de bord.

```
App.tsx → initDatabase() → getIsSetupComplete()  
├─ false → OnboardingScreen → markSetupComplete()  
└─ true  → Navigation à onglets (Dashboard / Historique / Appareil / Réglages)
```

4.2 Flux 2 — Connexion au distributeur

1. Le distributeur démarre, rejoint le WiFi, et **affiche son IP sur le LCD** pendant 5 s.
2. L'utilisateur ouvre l'onglet **Appareil**, saisit l'IP (pré-remplie si déjà mémorisée), tape **Connecter**.
3. HardwareContext.connect(ip) → httpService.setUrl() + setDeviceIp() (persistance) → httpService.connect().
4. L'app teste /status, **pousse l'heure** via /settime, passe l'état à « connecté », et démarre le **polling toutes les 3 s**.

4.3 Flux 3 — Distribution automatique d'une dose (le scénario clé)



Le point fort à retenir : la prise est enregistrée **automatiquement** dès l'appui sur le bouton physique, sans aucune manipulation dans l'application. Le tableau de bord affiche aussitôt la dose comme « prise » (badge vert).

4.4 Flux 4 — Modification du programme (Réglages)

1. L'utilisateur change les heures/le nom dans **Réglages** et tape « Enregistrer ».
2. `save(...)` écrit en SQLite, puis `scheduleDoseReminders(...)` reprogramme les 4 notifications.
3. Si le distributeur est connecté, `syncSchedule(...)` **pousse** le nouveau programme via `/sync` → le firmware met à jour ses variables, sauvegarde en **EEPROM**, et rafraîchit le LCD.
4. La nouvelle planification est donc cohérente sur les **trois** supports : téléphone, notifications système, et distributeur.

4.5 Flux 5 — Enregistrement manuel

Si l'utilisateur n'utilise pas le bouton physique (ou n'a pas le distributeur sous la main), il peut marquer chaque dose **Prise / En retard / Manquée** directement sur les cartes du tableau de bord. Même chemin de données : `logDose()` → `upsertDoseLog()` → SQLite → mise à jour des statistiques.

5 Nos avantages (points forts du projet)

5.1 Avantages techniques

1. **Architecture hybride autonome.** Le distributeur fonctionne **seul**, sans téléphone et sans WiFi (horloge de secours, EEPROM, planification embarquée). L'application est un *plus*, jamais un point de défaillance unique.
2. **Vraiment hors-ligne.** Aucune dépendance à Internet ni au cloud : toutes les données de santé restent sur le téléphone (SQLite). C'est un atout **confidentialité** majeur en contexte médical.
3. **Communication robuste et simple.** Le modèle de *polling* avec compteur `event_id` est sans état côté firmware, tolère les coupures réseau, et se reconnecte tout seul — idéal pour un microcontrôleur aux ressources limitées.
4. **Tolérance aux pannes matérielles.** LCD auto-détecté et facultatif, RTC avec repli logiciel, garde anti-brownout du moteur : le système dégrade proprement au lieu de planter.
5. **Résolution d'un vrai bug d'ingénierie.** Le diagnostic de la broche de strapping GPIO15 (qui empêchait le boot) démontre une vraie maîtrise du matériel ESP8266.
6. **Code typé et modulaire.** TypeScript de bout en bout côté app (types discriminés pour les événements), firmware découpé en modules réutilisables, requêtes SQL paramétrées (zéro injection).
7. **Multi-réseaux WiFi.** Le distributeur rejoint automatiquement le meilleur réseau connu, sans reflasher.

5.2 Avantages produit / UX

8. **Conçu pour les personnes âgées.** Grandes polices, mode confort visuel ($\times 1,32$), contrastes forts, gros boutons (≥ 56 px), code couleur vert/jaune/rouge, parcours en 2-3 taps.
9. **Enregistrement sans effort.** Un appui sur le bouton physique = prise journalisée automatique. Aucune compétence numérique requise.
10. **Rappels intelligents.** Notifications matin/soir + **relances 30 min après** si la dose n'est pas confirmée.
11. **Suivi d'observance clair.** Anneau de pourcentage, statistiques prises/retards/manquées, historique sur 30 jours et vue calendrier.
12. **Synchronisation tri-directionnelle.** Un seul réglage met à jour le téléphone, les notifications et le distributeur (EEPROM + LCD).
13. **Design « medical-grade ».** Palette clinique sobre, animations douces (fade/scale/spring), zéro encombrement visuel.

6 Questions du jury — et leurs réponses

Cette section anticipe les questions les plus probables, classées par thème.

6.1 Architecture et choix généraux

Q : Pourquoi avoir séparé le distributeur et l'application? R : Pour la **robustesse**. Le distributeur doit pouvoir distribuer les pilules même si le téléphone est éteint, déchargé ou hors WiFi. L'application enrichit l'expérience (suivi, historique, rappels) mais n'est jamais un point de défaillance critique. C'est un principe de conception « *fail-safe* » essentiel en santé.

Q : Pourquoi du WiFi/HTTP et pas du Bluetooth? R : Le HTTP sur WiFi local fonctionne avec un simple `fetch()`, **sans module natif**, donc directement dans Expo Go (pas de build natif). Il est aussi plus simple à déboguer (testable depuis un navigateur), plus stable sur la durée que le BLE, et ne nécessite pas d'appairage. Une première version utilisait le BLE; il a été entièrement retiré au profit du HTTP, plus fiable.

Q : Pourquoi React Native / Expo et pas du natif? R : Une seule base de code pour **Android et iOS**, un cycle de développement rapide, et un riche écosystème (SQLite, notifications, navigation). Expo permet aussi de tester instantanément sur un vrai téléphone via Expo Go.

Q : Pourquoi SQLite et pas un backend cloud? R : Confidentialité des données de santé, fonctionne **hors-ligne** garanti, latence nulle, et zéro coût d'infrastructure. SQLite est une base relationnelle complète, embarquée, éprouvée.

6.2 Le matériel et le firmware

Q : Comment calculez-vous la rotation du moteur pour un compartiment? R : Le 28BYJ-48 fait 4096 demi-pas par tour. Avec un réducteur 7,5 :1 entraînant un container à 15 compartiments, un compartiment = $4096 \times (7,5/15) = 2048$ demi-pas, soit exactement un demi-tour. Cette valeur est une simple constante recalculable si la mécanique change.

Q : Qu'est-ce que le pilotage en « demi-pas » et pourquoi ce choix? R : C'est une séquence de 8 micro-états (au lieu de 4 en pas complet) qui alterne l'alimentation d'une ou deux bobines. Cela double la résolution, augmente le couple et lisse la rotation, ce qui réduit les à-coups et le bruit.

Q : Que se passe-t-il si l'horloge RTC (DS3231) tombe en panne ou est absente? R : Le firmware bascule automatiquement sur une **horloge logicielle** : l'application envoie l'heure exacte via `/settime` à chaque connexion, et le firmware l'avance avec `millis()`. La fonction `rtcIsRunning()` empêche toute distribution tant que l'heure n'est pas fiable.

Q : Et si l'écran LCD est défectueux ou débranché? R : Le LCD est **auto-déecté** sur le bus I2C. S'il ne répond pas, tous les appels d'affichage sont ignorés et le système continue de fonctionner normalement (WiFi, HTTP, moteur). Cela évite qu'un afficheur défaillant bloque tout le démarrage.

Q : Vous avez parlé d'un bug de démarrage. Lequel? R : La 4^e bobine du moteur était câblée sur GPIO15 (D8), une **broche de strapping** de l'ESP8266. Le driver la maintenait légèrement haute au boot, plaçant la puce en mode arrêt : elle ne démarrait pas et ne rejoignait jamais le WiFi. Nous avons déplacé cette ligne sur GPIO16 (D0), une broche non-strapping. Diagnostic et correctif documentés.

Q : Pourquoi ne pas tester le moteur au démarrage? R : Le démarrage du moteur provoque un appel de courant (inrush) qui peut faire chuter la tension (*brownout*) et empêcher l'ESP8266 de booter correctement. L'auto-test est donc exposé **à la demande** via `/selftest`, jamais au boot.

Q : Comment gérez-vous les rebonds du bouton? R : Par un **anti-rebond logiciel** : tout changement d'état survenant moins de 50 ms après le précédent est ignoré. Seul un appui stable déclenche l'événement confirmé.

Q : Le programme survit-il à une coupure de courant du distributeur? R : Oui. Les heures et le nom du médicament sont sauvegardés en **EEPROM** avec un octet de validation (0xAB). Au redémarrage, le firmware recharge ces valeurs. Avec une DS3231 (pile CR2032), l'heure est aussi conservée.

6.3 La communication

Q : Comment l'application sait-elle qu'une dose a été distribuée ou confirmée? R : Par un système de **polling avec compteur d'événements**. Le firmware incrémente `event_id` à chaque événement et expose le dernier dans `/status`. L'app interroge `/status` toutes les 3 s; si `event_id` a changé, elle déclenche le traitement correspondant (afficher, journaliser...).

Q : Pourquoi du polling et pas du push (WebSocket, notification serveur)? R : Le polling est **sans état** côté firmware, trivial à implémenter sur un ESP8266, tolère naturellement les coupures réseau et la reconnexion. Un push temps réel serait plus lourd pour un microcontrôleur, sans bénéfice réel ici (un délai de 3 s est imperceptible pour l'usage).

Q : Que se passe-t-il si le WiFi tombe pendant une distribution? R : Rien de critique. Le distributeur distribue et journalise l'événement localement (`event_id`). Dès que l'app se reconnecte, elle relit `/status` et **rattrape** le dernier événement. Côté firmware, `maintainWifi()` reconnecte automatiquement au meilleur réseau connu.

Q : Comment gérez-vous les délais réseau / périphérique injoignable? R : Chaque requête a un **timeout de 6 s** (`AbortSignal.timeout`). En cas d'échec, l'app passe proprement en état « déconnecté » et affiche un message clair. Le polling s'arrête et reprend à la reconnexion.

Q : Qu'est-ce que CORS et pourquoi le firmware l'active-t-il? R : Le firmware envoie des en-têtes `Access-Control-Allow-Origin: *` pour autoriser les requêtes inter-origines, ce qui permet de tester l'API depuis un navigateur web et garantit la compatibilité avec la pile réseau de l'application.

6.4 La base de données et la logique applicative

Q : Comment évitez-vous les doublons dans le journal de doses? R : Grâce à la contrainte `UNIQUE(date, dose_type)` et au pattern **upsert** (`INSERT ... ON CONFLICT ... DO UPDATE`). Il y a au plus une entrée par dose et par jour; marquer à nouveau la même dose met à jour l'entrée existante.

Q : Comment est calculée l'observance? R : $\text{pourcentage} = (\text{prises} + \text{en_retard}) / \text{total} \times 100$. Une dose « en retard » compte comme prise (le médicament a été ingéré); seules les doses « manquées » pénalisent le score. Calcul agrégé en SQL sur 30 jours.

Q : Êtes-vous protégés contre les injections SQL? R : Oui, toutes les requêtes sont **paramétrées** (? + tableau de valeurs). Aucune donnée utilisateur n'est concaténée dans une chaîne SQL.

Q : Que se passe-t-il quand vous ajoutez une fonctionnalité nécessitant une nouvelle colonne? R : Des **migrations défensives** : on tente un `ALTER TABLE ADD COLUMN` dans un `try/catch`. Sur une base récente la colonne existe déjà (l'erreur est ignorée), sur une ancienne base la colonne est ajoutée sans perte de données.

6.5 L'expérience utilisateur et l'accessibilité

Q : Qu'avez-vous fait concrètement pour les personnes âgées? R : Mode « confort visuel » qui agrandit toute la typographie ($\times 1,32$), contrastes élevés, boutons d'au moins 56 px, icônes **toujours accompagnées de texte**, code couleur vert/jaune/rouge, parcours en 2-3 taps, et enregistrement automatique par bouton physique (zéro compétence numérique requise).

Q : Les rappels fonctionnent-ils sans Internet? R : Oui, ce sont des **notifications locales** planifiées par le système d'exploitation (déclencheurs quotidiens). Elles fonctionnent hors-ligne et même application fermée. Une relance est envoyée 30 min après chaque dose non confirmée.

Q : L'utilisateur doit-il ressaisir l'adresse IP à chaque fois? R : Non. L'IP est **mémorisée en SQLite** (device_ip) et pré-remplie. L'utilisateur n'a qu'à toucher « Connecter ».

Q : Et si l'utilisateur se trompe de statut (marque « prise » au lieu de « manquée »)? R : Il peut le corriger : l'upsert met à jour l'entrée du jour. Il existe aussi un bouton « Réinitialiser les prises du jour » dans les réglages (avec confirmation).

6.6 Sécurité, limites et perspectives

Q : Quelles sont les limites actuelles? R : La communication HTTP locale n'est pas chiffrée (acceptable sur un réseau domestique privé, mais améliorable). Le système suppose un seul utilisateur/distributeur. Le câblage physique final (Phase 1) restait à finaliser au moment de la rédaction.

Q : Comment sécuriser davantage la liaison? R : On pourrait ajouter un jeton d'authentification sur les endpoints POST, passer en HTTPS (TLS) ou utiliser un secret partagé. Sur un LAN domestique isolé, le risque reste faible.

Q : Quelles évolutions futures envisagez-vous? R : Retour haptique à l'enregistrement, gestion multi-médicaments et multi-doses, export PDF de l'historique pour le médecin, comptes multi-utilisateurs, détection de niveau de pilules restantes, et synchronisation chiffrée optionnelle.

Q : Comment testez-vous le système sans le matériel branché? R : Les endpoints /diag (scan I2C, état RTC/LCD, mémoire, RSSI) et /selftest (test moteur) permettent de diagnostiquer chaque sous-système. Côté app, l'enregistrement manuel et les écrans fonctionnent indépendamment du distributeur.

Q : En une phrase, quelle est la valeur de DisrtuCare? R : Un distributeur de médicaments **autonome et fiable**, doublé d'une application **accessible et hors-ligne** qui transforme une simple prise de pilule en un suivi d'observance clair et sans effort, pensé pour les personnes âgées.

7 Annexe — Référence de l'API HTTP

Endpoint	Méthode	Corps (JSON)	Action
/status	GET	—	État complet : programme, heure, dernier événement
/sync	POST	<code>{"am":"08:00","pm":"20:00","med":"Metformine","time":"14:30","rtc":true,"awaiting":false,"event_id":3,"last_event":{"evt":"confirmed","type":"AM"}}</code>	Met à jour le programme + EEPROM + LCD
/settime	POST	<code>{"h":14,"m":30,"s":0,"day":1,"month":5,"year":2016,"rtc":true}</code>	Régle l'horloge
/led	POST	<code>{"state":1}</code>	Allume (1) / éteint (0) la LED
/motor	POST	<code>{"slots":1}</code>	Fait tourner le distributeur de N crans
/diag	GET	—	Diagnostic matériel (I2C, RTC, LCD, mémoire, RSSI)
/selftest	GET	—	Auto-test du moteur (à la demande)

Exemple de réponse /status :

```
{
  "am": "08:00",
  "pm": "20:00",
  "med": "Metformine",
  "time": "14:30",
  "rtc": true,
  "awaiting": false,
  "event_id": 3,
  "last_event": { "evt": "confirmed", "type": "AM" }
}
```

Types d'événements :

evt	Signification
dispensed	Le moteur a tourné, la pilule est dans le bac (type = AM / PM / TEST)
confirmed	L'utilisateur a appuyé sur le bouton physique (type = AM / PM)
error	Défaut matériel; le champ msg le décrit